

IFT436 - Devoir #2

À remettre avant le **vendredi 2 octobre à 23h59**.
Tout retard entraînera jusqu'à 33% de pénalité par jour.

Pondération. Ce travail compte pour 6% des points de la session. Si votre note dépasse 100, elle sera rabaisée à 100.

Modalités. Considérez les points suivants.

- Le travail peut être fait seul ou en équipe de deux.
- Le devoir doit être remis en format pdf via turnin. La provenance de votre pdf n'a pas d'importance — ce pdf peut provenir d'un export d'un fichier Word ou d'un fichier texte, d'un scan de vos écrits/dessins, etc.
Il n'y a pas de norme de présentation particulière. Écrivez lisiblement, et assurez-vous seulement que vos noms et CIP apparaissent clairement sur le devoir remis.
- Lorsqu'une complexité en temps est demandée, nous supposons le pire cas, sauf mention contraire.
- Vous pouvez utiliser les techniques de preuves que vous désirez, les indices ne sont que des suggestions.
- Vous pouvez utiliser les résultats démontrés en classe et dans les solutionnaires d'exercices sans avoir à les démontrer. Par contre, si vous utilisez de nouveaux résultats, vous devez les démontrer.

Question 1 : un peu de grand-O (30 points)

Entraînez-vous un peu à classer les fonctions en taux de croissance.

a. Répondez par oui ou par non aux questions suivantes. Vous n'avez pas à justifier vos réponses (ceci sera fait au prochain numéro).

(i) Soient $f(n) = n^4 + 20n^2$ et $g(n) = (n^2/1000 + 1)^2$
Est-ce que $f(n) \in \Theta(g(n))$?

(ii) Soient $f(n) = \log n$ et $g(n) = \log_{10}(n^{436})$
Est-ce que $f(n) \in \Omega(g(n))$?

(iii) Soient $f(n) = (2n)!$ et $g(n) = n!$
Est-ce que $f(n) \in O(g(n))$?

(iv) Soient $f(n) = \log((2n)!)$ et $g(n) = \log(n!)$
Est-ce que $f(n) \in O(g(n))$?

(v) Soient $f(n) = \sum_{i=1}^n 2^i$ et $g(n) = 4^n$
Est-ce que $f(n) \in \Omega(g(n))$?

b. Choisissez **deux sous-questions de la question précédente** et démontrez vos réponses, en utilisant les définitions appropriées de O , Ω et Θ . Vous devez choisir au moins un de (iii), (iv) et (v).

Question 2 : le tri sélection (15 points)

Le *tri sélection* implémente la stratégie suivante pour trier un tableau T . On initialise d'abord un tableau de sortie S vide. Puis on répète jusqu'à ce que T soit vide: on trouve le minimum m de T , on enlève m de T et on ajoute m à la fin de S .

Donnez la complexité en temps de cet algorithme dans le pire cas avec la notation Θ . Justifiez votre réponse.

Remarque. À vous de choisir la structure de données utilisée pour représenter T et S . Vous pouvez utiliser sans preuve les complexités connues pour les structures de données standard.

Question 3 : détection de collision 1D (30 points)

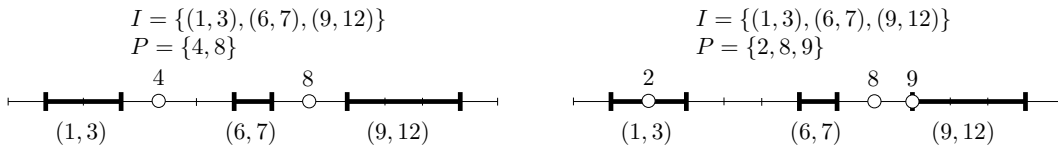
Vous voulez prendre rendez-vous avec un professeur qui a une liste d'intervalles de disponibilités. De plus, vous avez une liste de moments précis où vous pourriez prendre ce rendez-vous. Vous voulez savoir s'il y a un de ces moments qui entre dans ces disponibilités. Ce genre de problème survient dans de multiples contextes sous d'autres formes. On va donc l'abstraire en le formalisant comme suit.

Problème de la collision 1D

Entrée: un ensemble d'entiers $P = \{p_1, p_2, \dots, p_n\}$ représentant les points 1D, et un ensemble de paires d'entiers $I = \{(a_1, b_1), (a_2, b_2), \dots, (a_m, b_m)\}$ représentant des intervalles de segments.

Sortie: une paire $(p_i, (a_j, b_j))$, où $p_i \in P$ et $(a_j, b_j) \in I$ tels que $a_j \leq p_i \leq b_j$, ou bien *null* s'il n'existe pas de telle paire point/intervalle.

La figure ci-dessous donne une représentation visuelle du problème. Dans l'exemple de gauche, on retournerait *null*. Dans l'exemple de droite, vous pouvez retourner soit $(2, (1, 3))$ ou bien $(9, (9, 12))$. Donc on ne veut qu'une seule paire point/intervalle.



Répondez aux questions suivantes:

- L'algorithme naïf fera la chose suivante: pour chaque $p \in P$ et pour chaque $(a, b) \in I$, vérifier si p est contenu dans l'intervalle entre a et b (inclusivement), en retournant p et (a, b) si c'est le cas, et en retournant *null* si aucun point/intervalle n'a été trouvé. Quelle serait la complexité en temps de l'algorithme naïf, en terme de $|P|$ et $|I|$ en notation O ? Justifiez votre réponse.
- Supposons que $|P| = |I|$. Donnez le pseudo-code d'un algorithme pour ce problème qui atteint une meilleure complexité que l'algorithme naïf dans ce cas. Donnez la complexité O de votre algorithme avec justification. Vous n'avez pas à justifier que votre code est correct (mais il doit l'être).

Question 4 : détection de cycles (25 points)

Cette question concerne les graphes non-orientés.

- a. Donnez le pseudo-code d'un algorithme qui, étant donné un graphe $G = (V, E)$, détermine si G contient un cycle et, si c'est le cas, retourne les sommets d'un tel cycle (dans l'ordre qui forme le cycle).

Vous n'avez pas à justifier la complexité de votre algorithme, mais elle doit être aussi basse que possible.

- b. Un *unicycle* est un graphe dans lequel il est possible de retirer une arête pour obtenir un arbre. En d'autres termes, $G = (V, E)$ est un unicycle si $\exists e \in E(G) \cdot (G - e \text{ est un arbre})$, où $G - e = (V, E \setminus \{e\})$.

Dans vos mots¹, expliquez comment concevoir un algorithme qui détermine si un graphe est un unicycle. Donnez ensuite la complexité O en temps de cette algorithme.

¹Ceci veut dire que vous pouvez donner du pseudo-code, mais il est aussi acceptable d'expliquer votre algorithme en phrases, en énonçant seulement les étapes importantes.